



Wardenclyffe: Technical Design Document

This specification details the architecture for Wardenclyffe, an open-source, AI-native agentic framework built using Next.js, TypeScript, and Vercel. It functions as the underlying operating system for Latin American startups and SMBs, transforming traditional operations into an intelligent, self-regulating closed loop.

1. Executive Summary & Philosophy

Wardenclyffe provides the software engineering scaffolding required to eliminate human middleware and coordinate autonomous tasks without proprietary vendor lock-in.

- **Mental Prototyping (Tesla):** Agents generate complete execution specs and validation criteria *before* modifying data or running scripts.
- **Hierarchical Pattern Recognition (Kurzweil):** High-level business intents are continuously decomposed into granular, recognizable tasks.
- **Deterministic Harnessing (Osmani):** Agent tasks run inside strict software boundary layers. Success is silent; failures trigger automatic self-correction loops.

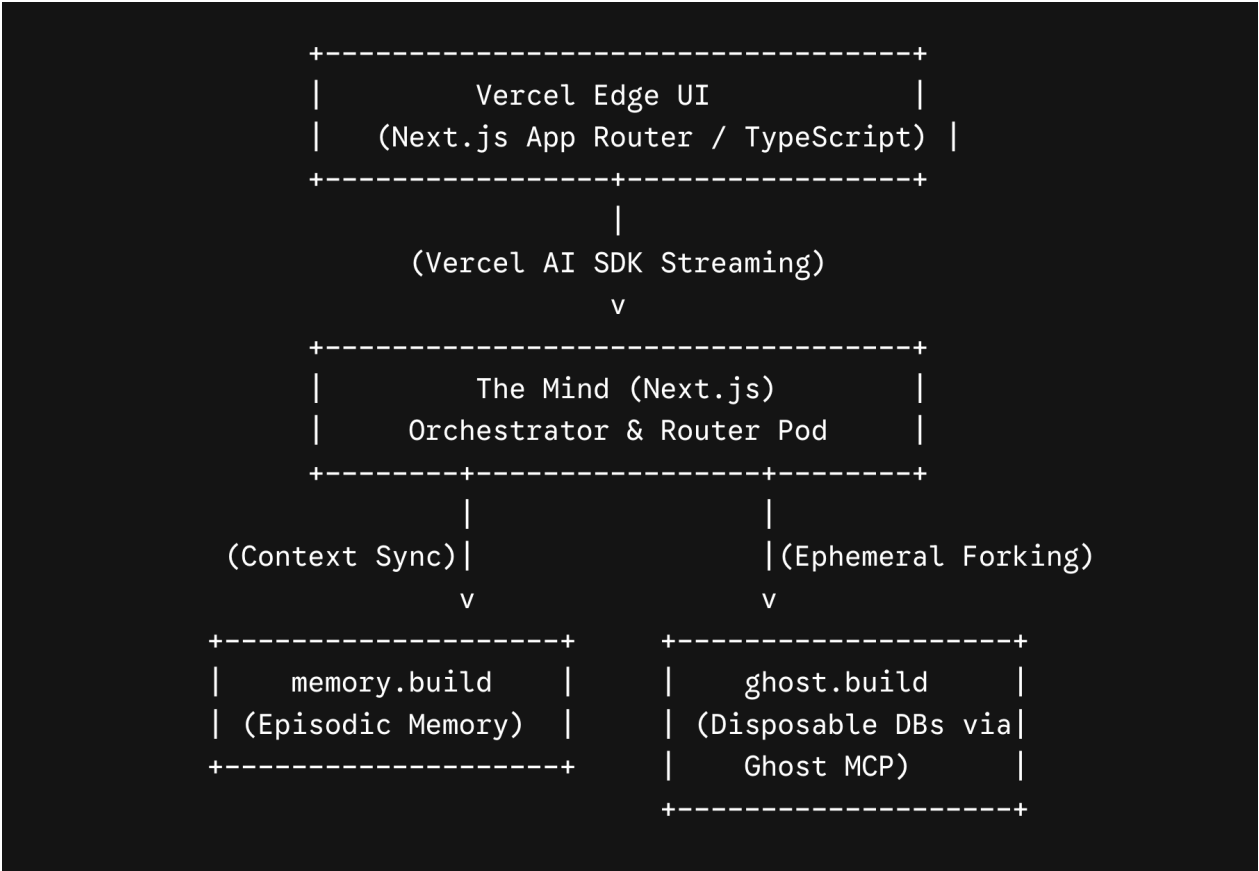
2. Technology Stack & Core Infrastructure

Layer	Technology	Role
Meta-Framework	Next.js (App Router)	Serverless edge runtime, application layout, and dashboard streaming.
Language	TypeScript	Type-safe data contracts between the frontend UI, schemas, and execution payloads.
Deployment	Vercel	Global edge-network deployment with native streaming support via the



		Vercel AI SDK.
Context & Ephemere	<u>memory.build</u>	Episodic and working long-term memory layer across distinct sessions.
Agent Database Substrate	<u>ghost.build</u>	Agent-first disposable, forkable Postgres infrastructure with native Model Context Protocol (MCP).

3. System Architecture





A. The Core Control Loop (Next.js Edge API Routes)

The central orchestrator resides within Edge-optimized Next.js Route Handlers. When an operator invokes an action, the **Planning Agent** models the target state as a structured JSON object. The system uses the **Vercel AI SDK** to stream internal steps, reasoning traces, and token costs directly back to the React UI.

B. Context & Memory Isolation (memory.build)

To maximize the efficiency of open-weight models (e.g., Llama 3, Mistral) without causing context-window collapse, Wardencllyffe offloads past interactions and corporate preferences to memory.build.

- Every conversation, automated report, and action artifact is captured into episodic graph segments.
- Before any execution agent executes a task, it dynamically calls the memory.build API to pull in the exact temporal and semantic facts required for the action, preserving the model's core tokens for reasoning.

C. Database Sandboxing & MCP Execution (ghost.build)

Autonomous agents can accidentally corrupt production data if left unconstrained. Wardencllyffe handles data manipulation through ghost.build.

- **Disposable Testing:** When a data or migration agent needs to operate, it tells Ghost to create an on-demand, virtual fork of the PostgreSQL database.
- **Native MCP Access:** The agent interacts through the built-in Ghost Model Context Protocol server. The agent safely runs SQL queries, inspects schemas, and verifies changes completely inside an isolated Postgres sandbox.
- Once the validation harness passes, the changes are flagged for human-in-the-loop review before final deployment.



4. Repository & Directory Layout

```
wardenclyffe/
├── app/                                # Next.js App Router
│   ├── api/                           # Vercel serverless edge routes for agents
│   │   ├── chat/                      # Orchestration streaming endpoint
│   │   └── harness/                  # Verification & feedback webhooks
│   └── dashboard/                    # Closed-loop tracking visualization
├── components/                        # React components for real-time trace viewing
├── lib/                               # Framework Core
│   ├── memory/                       # memory.build API Client integrations
│   ├── ghost/                       # ghost.build MCP tool configuration
│   └── orchestrator/                 # Hierarchical intent router
├── types/                            # Universal strict TypeScript definitions
├── wardenclyffe.config.ts             # SMB boundaries, allowed tools, and operational truths
├── package.json
└── tsconfig.json
```

5. Implementation Milestones

Phase 1: The Scaffolding (Next.js + TypeScript)

Build out the foundational App Router project structure on Vercel. Set up the stream pipelines using the Vercel AI SDK and map the type systems for agent-to-agent data handling.

Phase 2: Structural Memory & Sandboxing (memory.build & ghost.build)

Hook the Next.js API endpoints into memory.build for cross-session history



management. Add the Ghost MCP hooks so that a local or cloud-hosted open-weight model can spin up a Postgres database instance entirely via tool-calling.

Phase 3: The LatAm Toolbox

Establish local-first templates specifically tailored for regional SMB processes, allowing zero-overhead tool attachment.

For a practical demonstration of how agents interact with disposable sandboxed data layers, check out this guide on [Connecting Agents to Real Databases via Ghost](#). This video breaks down the step-by-step process of using an MCP server to let an AI write queries, test schema modifications, and seed databases inside isolated cloud infrastructure.